Quasi-Novelty Revisited: Continual Reinforcement Learning, Information Rewards, and the Geometry of Adaptation

Jacob Wilson
University of Southern Mississippi
jbw622@msstate.edu

October 13, 2026

Working manuscript — revision 0.1

Abstract

How can an agent, starting from random weights and learning through reinforcement learning alone, continually acquire useful behavior without destroying the conditions that made learning possible? Quasi-Novelty investigates this question through observation-derived information rewards, recurrent state, and interacting immediate and episode-duration credit. This working manuscript brings together the original command-discovery experiments and subsequent studies of reward design, entropy, observation coupling, and update geometry. The working hypothesis is that local adaptation and episode-scale reinforcement can organize shared representations while preserving access to useful behavior; the evidence does not yet establish this mechanism as sufficient or necessary.

The central advance is one from-random agent that executed 36 verified maximal pointer-following chains of depth two or three across 21 episodes and 26 distinct command sequences, without pretraining, demonstrations, or supervised command targets. The behavior was transient. Attenuating carried encoder offsets later restored its expression on selected recorded contexts without retraining persistent weights, separating local loss of access from complete erasure.

Acquisition remains exceptionally difficult to reproduce: seven completed fresh repeats produced no follows or chains at the frozen October 13 review. Historical audits and failed experiments are retained because they constrain the hypothesis. The next phase will seek additional compute for independent replication, controlled credit studies, and prospective retention tests. The program remains unfinished: reliable acquisition, sustained adaptation, and transfer to unfamiliar conditions are open.

1 Introduction

The original Quasi-Novelty paper reported command discovery within a longer investigation into sustained online learning: how can an agent keep acquiring behavior without collapsing into a terminal habit? The original experiments supplied no pretrained language model and no reward explicitly marking a command correct. They nevertheless produced repeated executable roots. The apparent simplicity of that result motivated a broader program: can local information-seeking behavior develop into dependent sequences, and can online learning preserve those sequences while the stream of experience continues? [E01]

The later experiments made us more careful about what counts as progress. An agent can collect varied errors, keep executing an uninformative command, or survive longer without

learning to use what it sees. An encoder may represent an observation while the decoder ignores it. Even aligned gradients can produce quite different movements once the optimizers act. We also found that a learned operation could disappear from behavior while remaining recoverable in the persistent weights. Much of this revision is about separating those cases.

The program's operational term *chaining* means dependent command sequencing across at least two distinct states. It can be reactive or habitual. A file-read output naming the next file, followed by an executed read of that file and then its successor, qualifies when the dependencies are verified. Such behavior does not by itself establish planning, lookahead, a world model, or general reasoning. The breakthrough reported here is the first execution-verified pointer chaining found in the inspected program, not a claim of priority over other research.

We report the progress of a changing experimental program here. There was no single preregistered comparison covering all the architectures. The historical audit, numerical qualification, completed training campaigns, exploratory discovery, and frozen checkpoint interventions each answer different questions, and we try to keep their conclusions separate. References [1–5] place the work alongside research on intrinsic motivation, copying architectures, and continual learning. Evidence identifiers [E01–E12] point to the project's underlying records.

1.1 The question behind the experiments

We want an agent whose education comes from its interaction history, we are still some way from showing that this can be sustained. There is no pretrained competence to fine-tune and no teacher specifying the correct command. Exploration must first bring useful regularities within reach. They then have to influence learning and remain available as learning changes the agent. Even a fixed environment does not give us a stationary stream: novelty is consumed, the policy changes what it encounters, and its representation of those observations changes too. A behavior can become an exhausted habit and still be the route to the next discovery.

The central *reconciliation hypothesis* is that immediate information-driven adaptation and slower episode-level reinforcement can organize the same encoder into a regime that supports both discovery and behavioral continuity. More specifically, immediate updates may change what subsequent recurrence preserves, suppresses, or makes retrievable, while episode credit organizes that adaptation over a longer horizon. Update frequency, update magnitude, and temporal credit horizon are separate variables. The stronger interpretation—that their interaction preserves or extends competence over long online histories—remains a hypothesis. Opposed updates may stabilize a useful regime, but opposition may also interfere, and aligned updates may either consolidate a skill or intensify a failure. Their functional effects must be measured.

We keep returning to four questions. What makes an intrinsic reward favor learnable structure rather than endlessly varied noise? When does information in an observation actually control the next action? How do immediate, delayed, and episode-duration credit interact in parameter space and recurrent state? And when a behavior vanishes, has it been erased, become inaccessible, or lost the experience that sustained it? These are questions about continual learning as much as acquisition. A long run, persistent memory, or one successful chain does not settle the retention problem.

1.2 Relation to prior work

Intrinsic motivation has been implemented through action-conditioned feature prediction [1] and prediction against a fixed random network [2], among other approaches. Their successes motivate learning from internal signals, while work on stochastic 'noisy TV' traps shows why prediction error is not automatically useful curiosity [3]. The generate-and-copy operation in the present decoder shares a mechanism family with pointer-generator networks [4]; the claim here concerns its acquisition through this RL interaction stream, not invention of copying. Continual-learning research frames the accompanying problem of retaining earlier knowledge while learning from changing data [5]. This study does not benchmark against those methods or claim superiority over them.

2 From the original result to stricter evidence

2.1 What survives the historical audit

The April manuscript described a 12,100-episode developmental trajectory and a headline validity above 95%. Recovery of the actual logs identifies the principal run as Exp 210, seed 42. It generated 336,845 actions through episode 12,100. The 95.248% statistic is the fraction of episodes containing at least one action admitted by a permissive prefix proxy; the corresponding action-level proxy fraction is 66.401%. Neither is an execution-success rate, and historical exit statuses are insufficient to reconstruct one. The reported 39,593 commands were the cardinality of a set populated from sampled commands, not the total number of actions. [E01]

There is a meaningful positive result beneath those corrections. In the first 1,000 episodes, with two-character commands, the agent repeatedly acquired the executable roots `dd`, `ld`, `id`, `df`, `ls`, and `ln`. Among 4,383 sampled proxy-positive outputs, 98.6995% appeared non-error under the retrospective screen. That selected sample supports root discovery; it is not a usefulness estimate for all generated actions. Three additional roots occurred once each, so the original nine-command count overstates the robust repertoire. [E01]

The actual historical quality signal multiplied bigram Jaccard observation distance by learned encoder displacement. It was not Jaccard alone. Command length increased by one every 1,000 episodes, which is a length curriculum without semantic demonstrations. The later audit also corrects the original architecture and termination description: the champion used 32 width-64 encoder latents and quality-band termination with a 200-step cap. These details are relevant because reward representation, action support, and episode boundaries all change the learning problem. [E01]

The early decoder-only figures likewise require restraint. A prominent 50,000-step TDE run's 29.352% score measured prefix admission; only 11 logged invocations of known commands exited zero in the forensic rescore. The blacklist figures were substantially affected by punctuation, parser behavior, and errors. Historical PCA trajectories remain descriptive, but a concentrated trajectory is not evidence of useful dynamics: in a later six-run encoder study, the gibberish failure had the highest encoder PC1 fraction, 0.9269. [E01, E02]

2.2 What subsequent measurements must establish

The revised evaluation separates exact executable-root selection, parsing, process completion, exit status, command-specific usefulness, physical state change, and acquired dependence on earlier outcomes. A zero exit status can reflect an empty-input utility; a state change can occur accidentally; a later successful read need not have been selected because of the earlier action. August qualification produced 686,476 records across 81 jobs, but usefulness, argument correctness, and causal-cascade labels were absent. Missing labels are not negative results. [E02]

The September work sharpened and operationalized the reconciliation hypothesis: frequent decoder-information updates and a separate episode-duration update both affect the encoder, potentially organizing behavior at different timescales. Persistent latents, associative state, carried low-rank offsets, and persistent parameter learning are distinct mechanisms. Replay under current parameters and offsets is also distinct from differentiating through the exact historical online trajectory. The subsequent geometry studies were designed to measure these distinctions instead of inferring them from survival or attractive latent plots. [E03, E04]

3 Information rewards and the implemented learning system

3.1 Novel content in a familiar form

The successful October agent used StructuredNovelty2 (SN2). Let T(o) be the set of distinct maximal byte runs matching `[A-Za-z0-9_.]+` in an observation, and s(o) a skeleton that maps letters to a common letter and digits to a common digit while preserving layout. A lifetime token memory estimates the fraction of tokens not seen before; a dictionary of recent skeletons estimates how familiar the form is under compression. In schematic notation,

$$r_t = N(o_{t+1})F(s(o_{t+1})), \qquad 0 \le N, F \le 1.$$

The implemented familiarity is one minus a clipped ratio of dictionary-conditioned to unconditioned compressed lengths, after a fixed overhead correction. The dictionary contains up to 64 recent skeletons and does not reset at episode boundaries. A bounded Bloom filter stores lifetime token membership; its false positives suppress novelty rather than creating it. Crucially, the emitted action's tokens enter memory before its output is scored. Simply echoing newly generated text cannot make those same tokens new to the agent. SN2 is a dimensionless compression-based proxy, not calibrated Shannon information or semantic understanding. [E05]

This construction expresses a useful preference—new content in recognizable structure—but does not make all unproductive novelty impossible. Familiar wrappers around fresh random payloads can still score. Environment structure therefore remains part of the method. A high reward establishes that the estimator paid, not that the agent acquired a useful causal relationship.

3.2 The winning architecture and environment

The discovery occurred in Chainlib v7b, a contained shell world with procedural fact files, renamed directories, and file contents that expose the name of a successor file. A local chain contains three distinct target files. The observed maximum depth of three is therefore an environment limit, not a measured planning horizon. World design supplies an accessible dependency structure; it must not be described as an unconstrained real-world discovery. [E06]

Table 1. Effective configuration of the discovery run; inherited configuration labels alone are insufficient.

Component	Implemented setting
Reference	FOSP seed 8501; learner/world/evaluation seeds 8501 / 108501 / 208501
Encoder	437,218 parameters; 16 slots × 64 dimensions; eight fresh and eight recurrent slots
Observation coupling	Even slots clocked in both halves; observation gain 4; cross-attention logit scale 4
Decoder	626,953 trainable parameters: base readout, hierarchical byte pointer, copy mixture, prefix GRU
Action support	95 printable ASCII characters; uniform lengths 2, 3, 4 during acquisition
Immediate credit	SN2 minus prior reward EMA (decay 0.99); encoder replay over 16 arrivals
Update settings	Small encoder SGD 1.0; Big encoder SGD 0.015; decoder SGD 0.1; native clipping
Extra decoder credit	Three previous command scores; decay 0.7; detached saved latents
Survival	Initial fuel 64; discounted SN2 replenishment; fuel ceiling 384
Allocation	65,536-action target at episode boundaries; 18-hour wall limit; one thread; 4 GiB worker cap

The decoder has two observation routes. It receives the encoder's normalized latent state, and it can directly copy printable bytes from the visible observation. A learned prefix GRU coordinates later characters with those already emitted. There is no hardcoded `pr` command template or pointer-following target supplied as a label. The decoder is trainable, despite a historical campaign directory name containing 'fixeddec'. The complete source-pinned reference, including installed adapters, is retained in E06.

Sampling uses Gumbel perturbations deterministically generated from a hash of the first 2,048 observation bytes and the previous command's token IDs. Conditional on that context, the learned scores, and the chosen command length, emission is deterministic. The categorical log-score used for learning is consequently a surrogate; it must not be presented as the exact conditional likelihood of independently resampled actions. The later length switch to four characters occurs at an episode start after 32,768 accumulated actions, after the entire observed chain window. [E06]

3.3 Three credit routes and a survival objective

For action t, immediate advantage is $A(t) = r(t) - b(t)$, where b is the pre-action reward EMA, followed by $b \leftarrow 0.99b + 0.01r$. The shared immediate loss trains the decoder and propagates through the reconstructed encoder trajectory. 'Small' names this immediate encoder credit route; it does not guarantee a numerically small update. An additional decoder-only trace assigns the same current advantage to recent command scores:

$$\mathcal{L}_{\text{trace}}(t) = -A_t \sum_{k=1}^{3} 0.7^k \log P_\theta(a_{t-k} \mid \bar{z}_{t-k}, \bar{c}_{t-k}).$$

The saved latents in this trace are detached. This route can reinforce earlier commands without backpropagating through their earlier encoder states. It is additional to the immediate update, not a replacement for it. 'Big' is a separate episode-end encoder update with duration advantage a = clip((T − L⁻)/L⁻, −5, 5), using the prior duration EMA. Its replay surrogate has the form −a Σ $g_j^T z_j$, with saved action-score sensitivities g_j held fixed. It is not an exact derivative through shell transitions or the full changing-parameter history. [E04, E06]

In the winner, one action burns one fuel unit and observed information replenishes it according to u(t) = 0.95^(t−1) min(16, r(t)/0.02). Information obtained earlier within an episode therefore buys more survival than the same information obtained later in that episode. Total possible replenishment is bounded by 16/(1−0.95) = 320; initial fuel 64 gives an accounting bound near 384 actions, below the configured 3,200-step safety cap. Episodes in the main analyzed prefix lasted 64–121 actions. No chain-completion label is used for termination. [E06]

Episode termination is part of the scientific objective. Duration means completed environment interactions, not wall-clock delay or extra internal computation. Changing what ends an episode changes what the duration update reinforces. Ending an episode, resetting the world, resetting latents, and resetting learned state are separate operations; a learning boundary need not erase every form of memory. Their interaction is central to the continual-learning hypothesis, and the present result does not identify a universally useful termination rule. [E03]

4 What reward and entropy studies established

4.1 A valid information measure can still support an unhelpful incentive

Several later failures help explain why replacing novelty with a more sophisticated scalar did not immediately solve the problem. Finite Bayesian reference studies separated surprise from posterior information gain, but their conclusions were model-relative. A mechanism absent from the hypothesis bank could be assigned to noise and receive little gain. In a separate predictive-response projection study, reducing late stochastic-channel payoff by about 66-fold still left stochastic channels earning 4.26–4.58 times more cumulative reward than deterministic maps across three matched seeds. These were estimator studies, not trained shell-agent successes. Bayesian correctness and desirable exploration incentives are separate requirements. [E07]

The native predictor portfolio supplies a complementary negative result. Thirty-two completed fits contributed 1,048,576 exact-prefix training actions, 1,159 reported-valid commands, and 57 filesystem changes. Their final evaluations recorded zero valid actions out of 2,050 in each of the frozen and predictor-online modes. Those modes are paired evaluations, not independent seeds. Some physical producer/consumer events occurred, including an assembler-created file later appearing in a listing, but neither learned ordering nor retention followed from their occurrence. [E07]

A direct byte-dependence reward also altered the command repertoire without meeting its prospective ordering screen in any of 12 completed fits. Several policies concentrated on one string; binary mixtures did not exceed shuffled-order dependence. That result concerns a

particular estimator, credit route, model, and budget. It is not a proof that information-derived objectives cannot support sequencing. [E07]

4.2 Entropy is not one quantity

Policy entropy measures uncertainty in scored character emissions; attention entropy measures concentration over eligible input locations; repertoire entropy describes an empirical distribution of emitted commands. Predictive codelength and posterior information gain are different quantities again. The original paper also confused units: a 97-symbol uniform distribution has entropy $\ln(97) \approx 4.575$ nats, or $\log_2(97) \approx 6.600$ bits. A value near 4.4 from a natural-log implementation is not 4.4 bits. [E01]

The historical 'zombie attractor' remains a useful descriptive name for sustained, diverse, unproductive behavior. It should not be promoted to a general theorem that entropy bonuses are harmful or that one decoder necessarily escapes. Seed, environment, decoder, reward, and budget were not fully matched in the original comparisons. Later experiments show both reward drought and persistent reward on invalid outputs; high entropy is not sufficient for discovery, and low entropy can mean either useful commitment or collapse. [E01, E07]

Table 2. Concentrated encoder attention did not identify the successful checkpoint. Same-fit, selected-context diagnostics; normalized first cross-attention-map entropy.

Checkpoint	**Observed regime**	**Normalized entropy**
Initial	Before training	0.4991
16k	Before chaining	0.2722
24k	Successful chain-period checkpoint	0.3518
32k	After behavioral collapse	0.2564

In the winner, both the unsuccessful 16k checkpoint and collapsed 32k checkpoint had more concentrated encoder attention than the successful 24k checkpoint. The pointer's raw attention also requires care: it can put most mass on nonprintable positions, while printable normalization gives a sharply selected target. The effective distribution used to emit a character is the relevant one. Attention concentration does not establish correct selection, and a heatmap alone does not establish either. [E08]

5 Geometry: how learning changes the agent

5.1 Measure optimizer movement, not just gradients

A major methodological correction preceded the useful geometric results. In an inventory of 219 registered fits, 48 had completed episode records available for the original diagnostic. Among those records, 149 global cosine entries across 17 fits, plus 489 group-level entries, lay outside the mathematical interval $[-1, 1]$. A tensor-only reproduction with very small parallel float32 vectors produced 12.405; scaled float64 geometry returned approximately 1. The missing historical vectors prevent retrospective repair. Those invalid measurements cannot support claims about alignment or cancellation. [E04]

The replacement instrumentation records actual Small and Big encoder displacements and verifies that they sum to the total parameter change. In six fresh diagnostic fits—three recipes and two seeds—every one of 1,530 episode movements telescoped exactly, and all 18

common-state intervention anchors restored by digest. The resulting measurements show why the distinction matters:

Table 3. Completed six-fit geometry study, approximately 16,384 actions per fit. Ranges are across fit-level medians, not confidence intervals.

Quantity	**Observed range**
Accumulated Small–Big gradient cosine	0.974–0.980
Actual Small–Big displacement cosine	0.10–0.17
Small share of encoder displacement	0.004–0.134
Cancellation fraction	0.011–0.082
Consecutive Small-step cosine	0.90–0.91

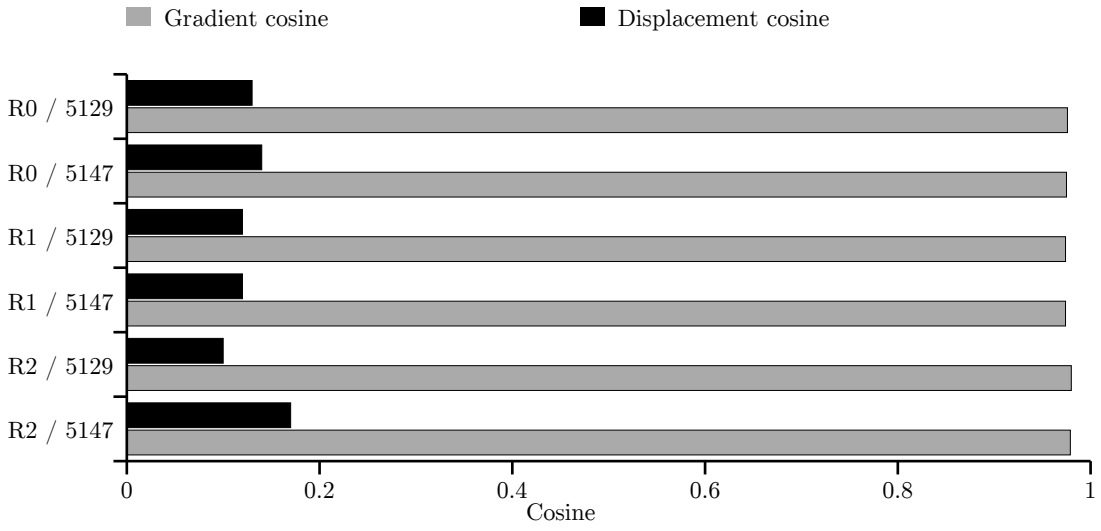


Figure 1. Gradient alignment and actual movement diverged under the two Adam optimizers. Six diagnostic fits; bars are fit-level medians from E04, without inferential error bars.

Independent Adam states transformed nearly parallel accumulated gradients into substantially different parameter movements. Small contributed as little as 0.4% of encoder displacement in one recipe, despite having a defined credit route. At the first common-state anchors, accumulated optimizer state suppressed Small roughly 10–100 times more than Big relative to fresh-moment steps. Local order effects were at most 0.002 under the study's normalized noncommutativity measure; cross-surrogate effects were small and not consistently interfering. All six runs ended near the fuel floor without retained useful behavior. These observations support a transmission problem in those recipes, not a general claim that Big cancels useful learning. [E04]

Follow-up studies also resisted a simple fix. Removing momentum improved encoder credit transmission in several comparisons, but slowing or delaying the decoder enough to raise encoder credit share to roughly 0.6–1.0 produced little useful behavior: all five such fits remained below the study's 7% late-validity screen. The decoder still had to learn. An earlier interpretation that a falling Fisher trace demonstrated loss of plasticity was withdrawn because declining trace also accompanied useful policy sharpening. Sensitivity, movement, and behavior must be evaluated together. [E04]

5.2 Representation is not the same as use

Splitting latents into fresh and recurrent slots improved observation encoding in a selected comparison without producing strong action dependence on the observation. At 24.6k actions in a single-seed comparison, an offline observation-readout probe had cross-validated $R^2 = -0.08$ in the base and 0.56 in the split model. Within the split model, measured command–state mutual information fell over training from 0.12 to 0.011 nats. The readout was an analysis tool, not a supervised training signal to the agent. Better representation alone did not establish a useful conditional policy. [E04]

Conversely, the later successful pointer routine did not require a distinct large-scale latent trajectory for every chain. In fixed-checkpoint replays of eight successful tapes, a common time-aligned path explained 98.82% of latent variation and 99.41% in recurrent slots. Clocked slots accounted for 97.63% of variation. Holding encoder input constant or fixing the clock still preserved correct commands on the selected diagnostic contexts when the decoder retained the true observation. A reusable latent state can drive a query that selects different targets through a direct observation-copy path. Stereotyped geometry is therefore compatible with useful reactive chaining. These replays are not recovered historical actor trajectories or held-out rollouts. [E08]

The design lesson is narrower than an architectural prescription. A model must have a route by which changing evidence can affect the selected action, but that route need not appear as high latent variance or concentrated encoder attention. The present system provides both an encoder route and direct byte copying. Controlled ablations are still needed to identify which routes are required to acquire the routine, rather than merely to express it after acquisition.

6 The breakthrough: dependent action from random initialization

6.1 Recorded behavior and its limits

The FOSP seed-8501 run completed 65,548 actions across 996 episodes. It produced 169 verified pointer follows and 36 maximal chains: 22 of depth two and 14 of depth three, covering 26 distinct command sequences in 21 episodes. The exact chain interval was actions 21,404–25,282. The terminal extension added no chains; its final audited 7,744-action, 121-episode tail contained zero executed actions under the audit classification. This is a transient discovery, not final-policy competence. [E06, E08]

A concrete witness is `pr T → pr J → pr s → pr l` at actions 21,439–21,442. The first output names J, reading J names s, and reading s names l. The agent must use a newly returned observation at each hop. Across episodes, the target names change. This goes beyond repeatedly issuing the same successful reader command, while remaining compatible with a reactive 'read the indicated file' operation rather than planning ahead. [E08]

Table 4. Stages in one discovery run. Action counts are global training indices; checkpoint labels refer to saved states near those counts.

Action / stage	**Observation**
65	First positive recorded information reward, before useful command acquisition
706	First logged valid command, od
6,513	First successful file read, pr j
11,884	First isolated pointer follow
16,438 checkpoint	Copy gate nearly open, but target score only about 5.1%
21,404	First depth-two chain
24,637 checkpoint	Precise target copying on selected contexts
25,282	Last verified chain
25,426	Directory error and near-unit Small encoder update
25,485 onward	Next episode loses valid execution; no chain recovery in original run
32,780 checkpoint	Native responses fail; offset attenuation restores local expression

The action-length denominator is essential. During the chain interval there were 429 pointer opportunities with a next action, but 264 drew only two or three characters and could not express `pr X`. Of 165 four-character opportunities, 161 produced a successful read and 157 followed the named target: 95.2% conditional on sufficient length, or 36.6% across all opportunities. In the preceding comparison interval, 11 of 170 four-character opportunities produced a correct follow (6.5%). These are retrospectively selected within-run windows, not held-out success rates. [E08]

Evidence for dependence comes from raw input/output witnesses, changing targets, a shuffled-pointer comparison, and intervention on the visible target. It should not be conflated with the stronger successive-hop-independence test: the registered statistic was 36 versus an expectation near 29.5, with $p = 0.135$. That test did not reject its null. A reusable observation-conditioned operation can compose into chains without showing an additional learned multihop advantage beyond its component steps. [E06]

6.2 A plausible learning process, with a concrete credit bridge

The initial reader habit repeatedly reached a file whose output exposed another filename. Thirty-five of 36 chains began immediately after the habitual `pr T` entry. Repeated known reads generally ceased paying novelty, but they continued to expose useful successor references. Six paid isolated follows were recorded before the first chain. A shared reader prefix and shared target selector could therefore accumulate reward across different literal filenames. The farm was an exposure route in this run; it was not sufficient, since other reader habits never developed correct targeting. [E08, E09]

For one character y, let $G(y)$ be the generator distribution, $C(y)$ the copy distribution, and g the copy gate. The scored mixture is $P(y) = (1-g)G(y) + gC(y)$. If $w = gC(y)/P(y)$, then the derivative of log P with respect to the gate logit is $w-g$. Positive advantage can increase copying when copying better explains the rewarded character. The copy query receives a responsibility-weighted gradient toward positions containing that character; it is not told which occurrence is semantically the pointer. Rewarded examples with different filenames can nevertheless reinforce shared selection features. This is an implemented learning route, not a reconstruction of the unsaved historical gradients. [E06, E08]

Delayed decoder credit provides a visible bridge across zero-immediate-reward actions. At action 21,405, advantage +0.558367 assigned trace coefficients +0.390857 to action 21,404 and +0.273600 to action 21,403. Those prior actions earned zero immediate reward. The coefficients follow exactly from the saved advantages and trace decay; they are not measured parameter displacement. Because this trace uses detached past latents, it establishes delayed decoder reinforcement, not that recurrent encoder backpropagation caused acquisition. That causal question remains central to the proposed next experiment. [E08]

6.3 What the trained mechanism does

At the successful checkpoint, the argument-copy mixture averaged 0.999975 and the selected target's categorical score was 0.999883, versus 0.01660 from the generator alone. Holding the original latent fixed and editing the visible target redirected 23 of 24 selected four-character commands. These interventions support a learned generate-and-copy operation: the generator supplies the reader prefix, prefix conditioning coordinates syntax, and copying supplies the changing argument. They do not establish that all those components were necessary during training. [E08]

Its robustness is limited and informative. On the same selected fixtures, plain two-line content retained 23/24 correct responses, whereas numbered content gave 0/24 and trimming trailing padding gave 4/24. Header-text removal gave 11/24 despite near-perfect conditional target selection, because command syntax failed. The behavior does not universally require the original printed layout, but it is sensitive to particular layout changes. All successful commands still used `pr`; generalization to other reader commands was not demonstrated. [E08]

7 Continual learning: acquisition, access, and retention

7.1 The reconciliation hypothesis in a successful interval

The chain-period update geometry provides a candidate connection between the two learning timescales. Across 27 episodes with positive duration advantage, median Small–Big displacement cosine was −0.562, with opposing directions in 24 episodes. Across 23 negative-duration episodes, median cosine was +0.954, with alignment in 21. A pooled average obscures this sign-conditioned structure. One hypothesis is that this opposition helped preserve a useful operating regime in longer episodes; the recorded directions alone do not establish whether Small was locally erosive or Big was protective. This is consistent with reconciliation, but remains an association in one changing trajectory. [E08]

Table 5. Sequential encoder-update geometry during the 50-episode chain period. Medians; these are actual movements, not common-state causal gradient comparisons.

Duration advantage	Episodes	Small norm	Big norm	Cosine	Cancellation
Positive	27	0.110	0.0148	−0.562	0.149
Negative	23	0.0143	0.0058	+0.954	0.009

Collapse followed a different regime. At action 25,426, `pr T` returned a directory error, with zero reward and advantage about −0.01128, alongside a nearly unit Small step. The episode's net Small movement was 1.494, compared with Big 0.015 and cosine near −0.023. Yet seven near-unit Small steps occurred before acquisition, and the same entry error had been survived many times. The step, episode-end credit, state reset, and changing experience co-occur. The record does not isolate one as the cause. [E08]

7.2 A lost behavior need not be an erased behavior

The most direct retention-related result comes from the collapsed 32k checkpoint. With persistent encoder and decoder weights unchanged, native carried offsets gave 0/24 correct responses on selected recorded pointer contexts. Scaling the effective offsets to 0, 0.125, or 0.25 gave 24/24 at each setting; the other tested scales, 0.5–1.25, returned to 0/24. Two completed failed repeat seeds remained at 0/24 with or without offsets. Offset removal therefore exposed retained local competence in the discovery run, but did not reveal the same operation at the sampled checkpoints of two failed repeats. [E08]

This separates two continual-learning failure modes: degradation of stored competence and loss of access through the current effective state. The intervention shows retained local competence after behavioral collapse. It does not establish sustained online recovery, broad resistance to forgetting, or retention after a distribution shift. The fixtures were selected from the successful run, command length was fixed at four, and the test did not resume the full learning loop. Same-fit continuation branches are distinct diagnostic experiments and do not count as fresh acquisition replications. [E08, E10]

We also need to ask what happens after the current behavior has been learned. Novelty memory changes what is rewarding, the actor changes its own data distribution, and carried offsets and persistent parameters can change which behaviors are expressed. Simply preventing further change would leave the harder problem untouched. We want the agent to acquire the next behavior while keeping earlier skills recoverable, within a finite memory and compute budget.

7.3 What a continual-learning claim will require

The next evaluation must measure acquisition, retention, recovery, and transfer separately on a continuing stream. Prospectively held-out targets and formats can test whether the same operation survives changes in surface form. Alternating previously encountered and unfamiliar environments can measure recovery cost and interference. Frozen tests reveal stored behavior, while paired online tests reveal adaptation under continuing updates. Useful outcomes include time to first acquisition, duration of competence, decline on earlier conditions, actions needed to recover, and whether new behavior improves without sacrificing old behavior. None of these can be replaced by a long episode, high reward, or a low-dimensional trajectory.

This places the work within the stability–plasticity problem of continual learning [5], while keeping the learning constraint explicit: the scientific agents begin from random weights and receive reinforcement learning signals alone. No pretrained teacher, transferred successful agent, supervised command labels, or corrective demonstrations are proposed for the next fresh-agent studies. The open question is how much structure the ongoing interaction and its intrinsic consequences can supply.

8 Why replication has been exceptionally difficult

At the frozen October 13 snapshot, seven prospective FOSP repeats had completed their approximately 65,536-action allocations. None produced a pointer follow or chain. Together they generated 458,932 actions and used 70.04 summed per-run wall hours on the shared machine; these are concurrent elapsed worker times, not measured CPU hours or sequential waiting time. Two additional repeats were running and seven were unstarted out of 16 registered. The ongoing and unstarted fits are not final failures. The original exploratory winner should not be pooled with these repeats to claim a measured 1/8 acquisition probability. [E10]

Table 6. Completed fresh repeats at the October 13, 01:56 CDT scorer snapshot. Pointer screens are exposure opportunities, not successes.

Learner seed	Training actions	Pointer screens	Follows	Chains
8511	65,559	0	0	0
8521	65,550	289	0	0
8531	65,565	13	0	0
8541	65,579	0	0	0
8551	65,556	0	0	0
8561	65,583	2	0	0
8571	65,540	4	0	0

The failures occur at different stages. In the October 12 matched-prefix audit, at 24,576 actions the winner had encountered 898 pointer screens, made 172 subsequent reads, and followed 146 targets. Eight started repeats had only 15 pointer screens combined at that same prefix. Most had not reached the regime where target learning could be exercised. However, exposure is not the whole explanation. A heterogeneous census of 49 from-random fits found 11 reaching at least 100 pointer screens, two exceeding a shuffled-copy null, and only the original FOSP producing chains. Some other recipes read after thousands of pointer screens while choosing the wrong file. These are confounded comparisons, useful for identifying failure stages but not for estimating a common success rate. [E09, E10]

Recreating an integer seed is also insufficient for bitwise replay. Printed timestamps enter the observation hash that generates sampling perturbations. Timestamp-only edits changed 46/48 commands at an early checkpoint, but 0/48 at the successful checkpoint. This demonstrates sensitivity before acquisition, not proof that timestamps caused the discovery. Early archives additionally lack complete RNG, environment, and novelty-memory state, and some historical checkpoint names were overwritten between seeds. New replications need complete provenance and exact state capture. [E01, E08]

The difficulty is that several things have to come together: discovering a reader, reaching informative screens, selecting the relevant character, having enough characters to express the command, and retaining a workable effective state. We have evidence for each as a plausible bottleneck, but still lack a minimal sufficient recipe. Longer runs and more hardware may let us observe a rare transition, they cannot be assumed to repair an unsuitable incentive or an unstable update.

9 Open experiments and the plan to secure compute

This manuscript is deliberately an interim account. The next phase will seek additional compute through university research allocations, research collaborations, and suitable sponsored compute programs. No new allocation is claimed here. The purpose is to obtain enough independent trajectories and controlled comparisons to distinguish rare acquisition from reproducible mechanisms, while instrumenting the transition densely enough to explain it.

9.1 Priorities after the current replication evidence is complete

Table 7. Proposed research questions and discriminating measurements. These are future studies, not results or a newly launched queue.

Question	Proposed comparison	What would change the interpretation
Does recurrent credit drive acquisition?	From-random 2 × 2: decoder trace on/off × encoder cross-arrival adjoints retained/detached; retain forward recurrence.	A repeatable acquisition difference with qualified forward equivalence and matched seeds.
Does update interaction preserve competence?	Match Small and Big interventions while recording actual movement and behavior before/after updates.	Retention benefit beyond exposure or step-size differences; not merely a negative cosine.
Can effective-state control prevent loss of access?	Prospective offset-management conditions with the same persistent-weight learning rules.	Sustained online behavior and later acquisition, not just rescue on selected saved contexts.
What prevents target selection?	Factor learner/world seeds; measure reader exposure, prefix score, gate, target mass, and syntax separately.	A mechanism that improves target choice after comparable exposure across seeds.
Does the operation transfer?	New target distributions, layouts, reader commands, and deeper dependencies; continued RL only.	Acquisition plus retention under changed conditions, with held-out evaluation.

One concrete proposed credit panel uses eight paired learner/world seed assignments in each of four conditions: 32 fresh fits at a nominal 65,536-action allocation, or 2,097,152 actions before episode-boundary overshoot. With the existing 18-hour per-fit ceiling, that is up to 576 worker wall-hours for the fits, with separately budgeted qualification, diagnostics, and storage. This is a planning envelope, not a power guarantee or a promised success probability. Its final allocation should be revised from the completed repeat distribution and prospectively chosen effect size. Ablations with two seeds are clues, not tests of architectural necessity. [E08, E10]

Additional capacity should buy both replication and explanatory resolution: dense complete checkpoints around acquisition and collapse; saved optimizer states, offsets, RNG state, reward memories, world snapshots, and exact observations; matched common-budget comparisons; and censored time-to-event analysis when a wall limit ends a run before acquisition. Completed negatives, interrupted runs, and unstarted cells must remain distinguishable. More configurations without these records would expand the search without resolving the hypothesis.

9.2 Compute efficiency is part of the scientific problem

The current workload is constrained by repeated recurrent differentiation and environment interaction, not simply by model size or available RAM. A local hardware audit found near-saturated CPU use while substantial memory remained available. Encoder-replay GPU prototypes show component-level gains, but an end-to-end gain for the scientific runner has not yet been established. One reported 1.4% whole-worker difference came from a single concurrent CPU-setting comparison and does not validate the GPU prototypes. Throughput claims must preserve numerical behavior, update semantics, and actual worker-level timing. [E11]

What we ultimately want is useful adaptation that continues beyond an initial training period, with limited interactions and compute. Acquiring competence from intrinsic consequences would only be the beginning. It would also have to survive changes in what is

rewarding and extend to new behavior without repeated supervised retraining. That would address a practical obstacle for adaptive agents. So far we have evidence about parts of this problem, and one brief instance in which several parts worked together. We do not yet have a reliable system.

9.3 Current working conclusion

The strongest completed finding is a learned observation-conditioned operation that composed into verified dependent behavior from random initialization using RL alone. The strongest accompanying limitation is its transience and failure to recur in seven completed prospective repeats. Geometry reveals how learning signals can fail to become useful parameter change; entropy and attention studies show why common diagnostics can mislead; offset interventions distinguish retained competence from access to it. The reconciliation hypothesis remains open. The next priority is a controlled improvement in acquisition probability, persistence, and transfer.

Appendix A. Historical evidence and unresolved transitions

This chronology retains positive, negative, and incomplete evidence. Its rows use different architectures, worlds, metrics, seeds, and budgets, so their numbers must not be pooled into a single success rate. 'Reported-valid' below retains the historical evaluator's meaning unless a stricter endpoint is specified. The separate supervised representation studies in adjacent project directories are outside this manuscript's RL-agent evidence.

Table A1. Selected historical record, March–October 2026.

Period / study	Recorded evidence	Interpretation and remaining question
Original Exp 210, seed 42 [E01]	336,845 actions through episode 12,100; episode prefix-proxy 95.248%; action prefix-proxy 66.401%.	Root discovery is supported; 95% command-execution success is not. Historical exit status is incomplete.
Initial two-character stage [E01]	28,290 actions; 14,767 prefix hits. Sampled positive outputs: dd 1,101; ld 1,044; id 993; df 838; ls 348; ln 56; three singleton roots.	Six repeatedly observed roots. The 4,383-output sample is selected, not a whole-run usefulness estimate.
Second long seed, Exp 211 [E01]	Length-seven action-proxy rate 0.00731%; length-eight rate 66.627%.	Collapse and recovery under a changing length curriculum; no isolated retention treatment.
Decoder-only follow-up [E01]	TDE flagship: 50,000 steps, 29.352% prefix admission, only 11 logged invocations of known commands exited zero.	A proxy can reward apparent acquisition without useful execution.
April–May weight PCA [E02]	84-run summary: mean PC1 0.669 versus 0.643 in contrasted descriptive groups; max/mean step ratios 1.87 versus 1.61.	Post hoc groups and scales confound a phase-transition interpretation.
May encoder PCA [E02]	Six selected runs: encoder PC1 0.8403–0.9269; decoder 0.6876–0.8672. Highest encoder concentration occurred in gibberish failure.	Geometric concentration alone does not establish useful learning.

Period / study	Recorded evidence	Interpretation and remaining question
June delayed-credit attempts [E02]	GAE: one 5,000-step update, then repeated aaaaaaaa; 5,600 total actions. n-step: stopped at 4,985 before its first update.	Insufficient evidence to test the reconciliation hypothesis cleanly.
August metric qualification [E02]	81 jobs, 686,476 action records; usefulness, argument correctness, and causal cascades unlabeled.	Measurement infrastructure; absent labels are not zero-success outcomes.
September short-sequence proxies [E03]	No-loop cd→use 0.60/episode versus heuristic 0.616; learned-loop 0.50 versus 0.511.	Chance-like adjacency proxy; use could be context-independent id.
September acquisition timing [E03]	57 completed runs: 20 acquired roughly 7–24% reported validity; first onsets 24.6k–53.2k, median about 41k.	Late seed-dependent onset; not a success-rate estimate for the October architecture.
September attention-tension test [E03]	12 completed fits across three seeds; registered chaining test failed on all three. Reward-only reduced self-entropy in two seeds while validity approached zero.	Optimizing an internal entropy statistic can move the metric without the target behavior.
September MoE and recurrence [E03]	MoE decoder prevented acquisition in its placement study; a later recurrent recipe acquired on three seeds but remained below its chaining target.	Architecture placement and trainable recurrence matter; syntax acquisition is a different endpoint from dependence.
October 7 predictor portfolio [E07]	32 completed fits; 1,048,576 actions; final 0/2,050 valid actions per paired evaluation mode.	Better prediction or a defined learning signal did not establish retained command behavior.
October 8 direct dependence [E07]	0/12 completed fits met the frozen ordering screen; several concentrated on one string or near-independent binary choices.	Repertoire restriction is not learned sequencing.
October 9 update geometry [E04]	Six fits; 1,530 telescoping episode records; 18 restored anchors. Gradient cosine 0.974–0.980, movement cosine 0.10–0.17.	Independent optimizers alter the functional delivery of credit. No retained useful behavior in this panel.
October 9 transmission follow-up [E04]	Five slow/lagged decoder fits: late reported validity 0.002–0.012 despite high encoder credit share.	A transmission improvement is not sufficient for acquisition.
October 9 split observation state [E04]	At 24.6k: base/split probe R^2 −0.08/0.56. Within the split model over training: command–state MI 0.12→0.011 nats.	Representation and policy use dissociate; diagnostic probe is not agent supervision.
October 12 FOSP discovery [E06, E08]	169 follows, 36 maximal chains, 26 sequences, 21 episodes; chain window 21,404–25,282.	First verified chaining in this program, from random weights using RL alone; transient, selected run.
October 12 same-fit interventions [E08]	Target edit redirects 23/24; collapsed native offsets 0/24, attenuated offsets 24/24 on selected fixtures.	Direct copy use and retained local access are supported; acquisition necessity and sustained recovery remain open.
October 13 replication snapshot [E10]	Seven completed repeats, 458,932 actions, zero follows/chains; two ongoing and seven unstarted.	No independent acquisition replication yet at this cutoff; incomplete runs remain separate.

This record explains why the program has shifted from asking whether a novelty signal can produce plausible commands to asking how experience becomes a durable, extensible behavioral organization. The historical failures do not erase the positive observations; they identify which earlier interpretations were too broad and which experiments would now be informative.

Appendix B. Evidence, reproducibility, and manuscript status

Status: working manuscript, revision 0.1. Scientific interpretation and replication remain in progress. The original run's terminal count is included, while detailed geometry and checkpoint studies retain their explicitly stated earlier prefixes. The repeat snapshot is frozen at October 13, 2026, 01:56 CDT; it is not a live status report. Counts from later ongoing work may change in a subsequent revision.

This revision synthesizes existing research and a read-only replication audit. It does not add training runs. Numerical statements are tied to original logs, source definitions, or retained analyses in the evidence index. Historical claims are corrected when the recovered source contradicts the earlier paper. No public dataset or code release is claimed; the companion evidence index records local paths and file hashes for author review. A public supplement should preserve the executable method, configuration, outcomes, and counterevidence before any broader reproducibility claim.

[E01] Original manuscript and forensic reconstruction. April Quasi-Novelty PDF; work/CLAIM_LEDGER.csv (especially C002–C025), champion_proxy_by_stage.csv, champion_proxy_summary.csv, and exp210_jsonl_summary.json in the i-x20 archive. Controls historical metrics, seed identity, reward, and restart limits.

[E02] Historical geometry, horizon, and metric repair. i-x20 archive: decoder_only/PCA_SWEEP_ANALYSIS.md; inter_step_v1/PCA_ANALYSIS_20260522.md; work/POLICY_HORIZON_AUDIT.md; work/paper/tables/FORENSIC_CORRECTION_TABLE.md and STAGE0_ENDPOINT_LADDER.md.

[E03] Reconciliation direction and September studies. perceiver_moe_refresh_validation: lm_research_20260922/RECONCILIATION_DIRECTION.md; reconciliation_campaign_20260923/README.md; reconciliation_followup_20260925/EVIDENCE.md. recon_research/FINDINGS.md: F31, F40, F42–F43. These are different recipes and endpoints.

[E04] Encoder geometry and observation coupling. encoder_reconciliation_geometry_2026-10-08/DESIGN.md, diagnostic_report_01.md, existing_01/numerical_audit.json, and vector_math_01.json. Follow-up interpretation: FINDINGS.md F141–F151, including the withdrawn F143 mechanism and F145 correction.

[E05] StructuredNovelty2 implementation. takeover_sweep_2026-10-06/sources_01/impl/sn2_x66.py. Defines token membership, action folding, skeleton compression, dictionary window, and episode persistence.

[E06] Winning configuration and behavioral verification. reviews/chaining_monograph_2026-10-12/reference_fit.json and final_run_count.json; reviews/fosp_chain_verify_2026-10-12/SUMMARY.md and repeat/ablation protocols. Effective source includes v8f/decoders_b.py, oc_adapter.py, decrew2_adapter.py, length_sched.py, and the pinned immediate/episode credit sources.

[E07] Information-reward and negative portfolio evidence. causal_head_2026-10-06/bayes_predictive_01/interpretation_01.json; reviews/2026-10-07-predictor-aggregate32-conclusions-03.json; reviews/2026-10-08-status-1756-bigplan-start.json. Detailed context in FINDINGS.md, October 6–8

evidence sections.

[E08] Breakthrough mechanism, geometry, and access interventions.
reviews/chaining_monograph_2026-10-12/REPORT.md, learning_pressure.json, credit_route_clarification.json, phase_update_audit.json, format_summary.json; reviews/chaining_conditions_2026-10-12/offset_sweep.json and timestamp_probe.json; reviews/fosp_deep_study_2026-10-12 and its latent_episodes_01 analyses.

[E09] Exposure, reading, and targeting across recipes.
reviews/chaining_conditions_2026-10-12/stages.json; reviews/clue_census_2026-10-12/SUMMARY.md. Matched prefixes and heterogeneous recipe census have different denominators.

[E10] Frozen prospective-repeat audit. reviews/manuscript_replication_audit_2026-10-13.json and journals/2026-10-13-manuscript-repeats-0158.log. The JSON preserves individual results, terminal records, source paths, hashes, and the distinction between completed, ongoing, and unstarted fits.

[E11] Resource constraints and acceleration limits.
reviews/performance_audit_2026-10-13/REPORT.md; performance_acceleration_2026-10-13/README.md and claude_ab_review_01.json. Component benchmarks and whole-worker comparisons are explicitly distinct.

[E12] Evidence manifest for this revision. evidence_index.json beside the manuscript records the local inputs and SHA-256 hashes used for review. The manuscript does not reproduce full model checkpoints or all training tapes.

Unless identified as the older i-x20 or perceiver archive, evidence paths above are relative to the recon_research project. Repeated analysis of the same run is not independent evidence. Recorded events, frozen interventions, proposed causal explanations, and future studies remain separately labeled.

References

[1] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven Exploration by Self-supervised Prediction. ICML, 2017. https://arxiv.org/abs/1705.05363

[2] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by Random Network Distillation. arXiv preprint, 2018; ICLR, 2019. https://arxiv.org/abs/1810.12894

[3] Augustine N. Mavor-Parker, Kimberly A. Young, Caswell Barry, and Lewis D. Griffin. How to Stay Curious while Avoiding Noisy TVs using Aleatoric Uncertainty Estimation. arXiv preprint, 2021. https://arxiv.org/abs/2102.04399

[4] Abigail See, Peter J. Liu, and Christopher D. Manning. Get To The Point: Summarization with Pointer-Generator Networks. ACL, 2017. https://arxiv.org/abs/1704.04368

[5] German I. Parisi et al. Continual Lifelong Learning with Neural Networks: A Review. Neural Networks 113, 54–71, 2019. https://arxiv.org/abs/1802.07569

Quasi-Novelty: Emergent Syntax Discovery via Observation Diversity in Minimal Reinforcement Learning Agents

Jacob Wilson
Mississippi State University
jake.b.wilson.1337@gmail.com

April 2026

Abstract

We investigate whether a minimal reinforcement learning agent, trained with no supervision, pretraining, or explicit reward for correctness, can discover valid Unix shell syntax purely through a pressure to produce diverse observations. Our reward signal, based on Jaccard distance over character bigrams of successive shell outputs, contains no notion of validity—yet valid commands emerge as a robust side effect of diversity maximization, because they produce categorically richer environmental feedback than invalid ones. We term this phenomenon *quasi-novelty*: the agent appears to explore meaningfully, but its objective is a shallow structural proxy that nonetheless induces deep behavioral organization.

We document a pathological failure mode called the *zombie attractor*—a stable equilibrium in which additive entropy regularization causes the agent to generate diverse but meaningless output indefinitely, achieving high reward without discovering any structure. We show that this attractor is not a property of the agent alone but arises from the interaction of three factors: entropy bonus magnitude, environmental error diversity, and decoder architecture. Critically, we identify an architectural mechanism we call *entropy self-correction*—a bidirectional coupling in controlled differential equation (CDE) decoders that creates oscillatory entropy dynamics, enabling escape from the zombie attractor through repeated collapse-recovery cycles.

Our experiments span two phases: an encoder-decoder architecture (136K parameters) achieving 95%+ valid command rates across a 9-stage developmental trajectory over 12,100 episodes, and a systematic decoder-only investigation (7–21K parameters) isolating the contributions of architecture, environment, and training algorithm. Principal component analysis of weight trajectories reveals qualitatively distinct phase transition signatures—sharp L-turns in memory-augmented decoders versus smooth curves in CDE decoders—providing geometric evidence for discrete versus continuous learning dynamics. We find that diverse shell error messages act as an implicit entropy regularizer, enabling valid command discovery *without* entropy collapse, while explicit entropy bonuses are not merely unnecessary but actively harmful. This work is part of a larger ongoing investigation into observation-grounded open-ended learning.

1 Introduction

Consider the following question: can a reinforcement learning agent, given no prior knowledge of language, no pretrained representations, and no reward signal for producing correct output, learn to generate syntactically valid commands in a complex environment? The answer, perhaps surprisingly, is yes—provided the environment itself is sufficiently structured to make valid behavior *more interesting* than invalid behavior, and provided the agent is equipped with nothing more than a pressure to produce diverse observations.

This paper presents a systematic investigation of this phenomenon across two experimental phases. In the first phase, we constructed a 136,358-parameter encoder-decoder system built from continuous neural ordinary differential equations, equipped with a fast-weight memory encoder and a controlled differential equation decoder. Trained purely via REINFORCE on an observation novelty signal derived from Jaccard distances over character bigrams of shell output, this system achieved over 95% valid command rates, discovered 9 unique shell commands, and exhibited a rich 9-stage developmental trajectory spanning 12,100+ episodes—all without any reward for command validity.

The success of this system raised a natural question: which components are actually necessary? The encoder processes shell output through a sophisticated continuous-time recurrence; the decoder generates characters through an autonomous ODE modulated by character feedback. Is the encoder essential, or does the decoder alone suffice? This question motivated our second experimental phase: a systematic investigation of decoder-only architectures spanning five neural differential equation variants (ODE, SDE, CDE, TDE) and a Transformer baseline, tested across whitelist and blacklist shell environments with varying error diversity.

The decoder-only experiments yielded several unexpected findings. First, valid commands emerge even without any encoder processing—a standalone CDE decoder with 17K parameters achieves 27–36% validity in diverse environments. Second, the interaction between environment and architecture matters more than either alone: the same decoder that permanently fails in a sparse-error environment succeeds easily in a diverse-error one, and vice versa for entropy-regularized variants. Third, we discovered that error diversity acts as an implicit entropy regularizer—in diverse shell environments, valid commands emerge *without* entropy collapse, maintaining character entropy near its maximum ($H \approx 4.2$ out of 4.4 bits) while achieving 27%+ validity. This stands in sharp contrast to sparse environments, where entropy must collapse to $H \approx 1.8$ before any valid commands appear.

These findings are unified by two central concepts. The first is *quasi-novelty*: the observation that a shallow, computationally cheap diversity metric (bigram Jaccard distance) can serve as a sufficient reward signal for discovering complex behavioral structure, because the structure of the environment creates a natural gradient from diverse-but-invalid toward diverse-and-valid. The name reflects a duality—the agent's novelty-seeking behavior produces genuine discovery, but the novelty signal itself is quasi in the sense that it measures surface-level structural variation rather than semantic understanding. The second is the *zombie attractor*: a stable equilibrium in entropy-regularized systems where the agent achieves indefinite survival by generating diverse gibberish, collecting entropy reward without ever discovering valid structure. The zombie attractor is not merely a training failure but a formally characterizable dynamical regime with specific entry conditions, maintenance mechanisms, and—under certain architectures—escape dynamics.

This work is part of a larger, ongoing investigation into observation-grounded open-ended learning in minimal agents. We report results as they stand, including incomplete ablations and seed-dependent outcomes, in the interest of documenting phenomena that may be useful to the broader community.

2 The Jaccard Novelty Formulation

The core of our approach is a reward signal that measures how different the current observation is from recent observations, using Jaccard distance over character bigrams. This section presents the formulation in detail.

2.1 Bigram Set Construction

Given a shell output string o_t (the environment's response to the agent's command at step t), we define its bigram set as:

$$\mathcal{B}(o_t) = \{(o_t[i], o_t[i+1]) \mid i = 1, \ldots, |o_t| - 1\} \tag{1}$$

where each element is an ordered pair of consecutive characters. We use character bigrams rather than unigrams or higher-order n-grams for several reasons: unigrams lose all sequential structure, trigrams and above create sparse sets that degrade Jaccard sensitivity, and bigrams provide a balance between local structural information and computational efficiency. The resulting sets are bounded in size by $|o_t| - 1$ but typically much smaller due to repeated bigrams in natural shell output.

2.2 Jaccard Distance over a Sliding Window

We maintain a sliding window $\mathcal{W}_t$ of the W most recent observations $\{o_{t-W}, \ldots, o_{t-1}\}$. The observation novelty at step t is defined as:

$$q_{\text{nov}}(t) = \left(\frac{1}{|\mathcal{W}_t|} \sum_{j \in \mathcal{W}_t} \left(1 - \frac{|\mathcal{B}(o_t) \cap \mathcal{B}(o_j)|}{|\mathcal{B}(o_t) \cup \mathcal{B}(o_j)|} \right) \right)^p \cdot \|\Delta z\|_2 \tag{2}$$

where p is a power parameter (default $p = 1$) that controls the sharpness of the novelty signal, and $\|\Delta z\|_2$ is the mean L2 displacement of the encoder's latent state (set to a constant in decoder-only experiments). The Jaccard distance term ranges from 0 (identical bigram sets) to 1 (disjoint bigram sets), and measures how structurally different the current output is from recent history.

2.3 Why Jaccard Distance Works as a Reward Signal

The key insight is that valid shell commands produce categorically different output than invalid ones. When the agent generates an invalid command, the shell returns an error message. In a whitelist environment (where unauthorized commands return "command not allowed"), 99.6% of error responses are identical, yielding near-zero Jaccard distance from the window. In a blacklist environment (where the real shell processes commands), errors are more diverse (41% "not found," 45% syntax errors, 11% other), but valid commands still produce output with substantially higher Jaccard distance from the window.

Concretely, valid commands produce Jaccard distances of approximately 0.8 from the recent window, while invalid commands produce distances of approximately 0.2—a 4× differential that REINFORCE amplifies within approximately 30 episodes. This differential exists because valid commands produce *novel* output (directory listings, system information, file contents) while invalid commands produce *repetitive* error messages. The agent never receives any signal about whether a command is "valid"—it simply discovers that certain character sequences produce more diverse observations.

2.4 Window Size

The window size W controls how far back the agent remembers. At $W = 5$, even moderately variable error messages provide enough novelty to sustain exploration without discovering valid commands. At $W = 50$, the bar for novelty is so high that even valid commands may not suffice. We use $W = 10$ for the encoder-decoder experiments and $W = 5$ for the decoder-only experiments, though we note that window size was not systematically ablated across both whitelist and blacklist environments—an important limitation we discuss in Section 12.

2.5 Training with REINFORCE

The policy gradient is computed via REINFORCE with an exponential moving average baseline:

$$\nabla_\theta J = \mathbb{E}\left[\sum_{t=1}^{T} (q_{\text{nov}}(t) - b_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] \tag{3}$$

where b_t is the EMA baseline (advantage $= (r - \text{EMA})/\sigma$), and T is the episode length. The baseline is critical: without it, raw REINFORCE produces 0% valid commands across all configurations tested.

3 Architecture

Our investigation spans two architectural phases: a full encoder-decoder system, and a systematic study of decoder-only variants.

3.1 Phase I: Encoder-Decoder (136K Parameters)

The full system consists of a continuous fast-weight ODE encoder (65,796 parameters) and a controlled differential equation decoder (70,562 parameters).

Encoder. The encoder processes shell output through a continuous-time recurrence. At each step, the raw observation is embedded and used to update a set of 8 persistent latent vectors $z \in \mathbb{R}^{8\times d}$ via:

$$\frac{dz}{dt} = f_\theta(z, t) + W_{\text{fast}} \cdot z \tag{4}$$

where f_θ is a learned drift function and W_{fast} is a fast-weight associative memory matrix updated via Hebbian learning with learned decay ($\tau = 0.79$, carry $= 0.73$ at convergence). Each latent has independent per-latent rotary position embeddings (RoPE), and the ODE is solved with an adaptive step-size solver (typically 8 steps at convergence). The fast-weight matrix carries approximately 73% of context from the previous step, creating a form of cumulative associative memory.

Decoder. The CDE decoder generates commands character-by-character from a 97-token printable ASCII vocabulary. Its dynamics are governed by:

$$\frac{dh}{dt} = f_\theta(h) + g_\theta(h, c_{\text{prev}}) \tag{5}$$

where h is the hidden state, f_θ is an autonomous drift (the ODE component), g_θ is a character-feedback function that injects information about the previously generated character, and c_{prev} is the embedding of the last character. The decoder receives context from the encoder via cross-attention with 4 heads. Episode termination occurs when decoder entropy drops below a threshold (indicating collapse) or reaches a maximum step limit. Command length grows via curriculum: +1 character every 1,000 episodes.

3.2 Phase II: Decoder-Only Variants (7–21K Parameters)

To isolate the decoder's contribution, we tested six architectures operating without any encoder, using a fixed random latent vector as context:

1. **ODE** (7,490 params): $dh = f_\theta(h) \cdot dt$. Pure deterministic continuous trajectory. No anti-collapse mechanism.

2. **SDE** (8,546 params): $dh = f_\theta(h) \cdot dt + \sigma(h) \cdot dW$. ODE with learned diffusion noise. The learned noise proved insufficient to drive exploration.

3. **CDE** (16,866 params): $dh = f_\theta(h) \cdot dt + g_\theta(h, c) \cdot dc$. ODE plus character feedback—the bidirectional coupling that enables entropy self-correction.

4. **TDE** (20,964 params): CDE plus a fast-weight memory $h \leftrightarrow m$ (a $d \times d$ matrix with Hebbian read/write). Tridirectional coupling: ODE dynamics, character feedback, and memory.

5. **Transformer** (16,738 params): Standard causal self-attention over generated characters. Discrete attention baseline.

6. **LSTM** (14,850 params): LSTMCell with character embedding input. Discrete gating memory, comparable to TDE.

All decoders generate 2-character commands (fixed `cmd_len` $= 2$), use $d_{\text{model}} \in \{32, 64, 128\}$, and are trained with REINFORCE on observation novelty at $\alpha = 0$ (no entropy bonus) unless otherwise noted.

4 The Zombie Attractor

When entropy regularization is added to the reward signal, a pathological equilibrium emerges that we call the *zombie attractor*—a stable dynamical regime in which the agent generates diverse but structurally meaningless output, collecting entropy reward without discovering any environmental structure. This section characterizes the zombie attractor in detail.

4.1 Definition and Conditions

Consider the standard entropy-regularized reward:

$$r_t = q_{\text{nov}}(t) + \alpha \cdot H(\pi_t) \tag{6}$$

where $H(\pi_t)$ is the Shannon entropy of the character distribution at step t and $\alpha > 0$ is the entropy bonus coefficient. The zombie attractor is the equilibrium in which the agent maximizes r_t primarily through the $\alpha \cdot H(\pi_t)$ term, maintaining high character diversity (and thus high entropy reward) without concentrating on character patterns that would produce valid commands.

Three conditions are jointly necessary and sufficient for the zombie attractor:

1. **Entropy bonus dominance** ($\alpha > 0$): The entropy term must be present and sufficiently large relative to the quality differential between valid and invalid commands.

2. **Environmental error diversity**: The shell environment must produce sufficiently diverse error messages that even invalid commands generate moderate observation novelty.

3. **Insufficient quality gap**: The novelty differential between valid and invalid commands must be insufficient to overcome the combined entropy bonus and error-derived novelty.

Removing *any one* of these conditions breaks the zombie attractor. Setting $\alpha = 0$ eliminates condition 1. Reducing error diversity (e.g., whitelist mode) weakens condition 2. Increasing the quality gap (e.g., longer training) can overcome condition 3.

4.2 Experimental Evidence

We ran the full encoder-decoder architecture across 5 seeds (42, 137, 7, 256, 1337) at both $\alpha = 0$ and $\alpha = 4.0$ for 300 episodes each.

Table 1: Zombie attractor formation across seeds. PT = phase transition (valid rate > 1%).

Seed	α=0 H_f	α=0 Valid%	α=4 H_f	α=4 Valid%	α=0 PT	α=4 PT
42	1.60	0.10%	2.98	0.14%	No	No
137	1.63	0.02%	2.65	0.06%	No	No
7	**0.54**	**19.8%**	**3.07**	**0.02%**	**Yes**	**No**
256	**0.72**	**19.6%**	**1.64**	**29.2%**	**Yes**	**Yes**
1337	2.15	0.04%	3.13	0.09%	No	No
Mean H_f (non-PT seeds)			α=0: 1.79	α=4: 2.92 (**63% higher**)		
PT count			α=0: 2/5	α=4: 1/5		

Seed 7 provides the clearest demonstration: at $\alpha = 0$, the agent transitions to 19.8% validity with entropy collapsing to $H = 0.54$. At $\alpha = 4.0$, the same seed is trapped in the zombie attractor—0.02% validity with entropy maintained at $H = 3.07$. The entropy bonus keeps the agent generating diverse random characters, collecting approximately 97% of its reward from entropy rather than observation novelty.

Seed 256 demonstrates that the zombie attractor is a *delay mechanism*, not an absolute barrier. This seed escapes at $\alpha = 4.0$, reaching 29.2% validity—but the transition is delayed relative to $\alpha = 0$. The initial quality differential from valid commands was strong enough at this seed for REINFORCE to gradually overcome the entropy bonus.

4.3 Episode Length as Phase Transition Marker

A striking signature of the zombie attractor is its effect on episode length. Before the phase transition, episodes are short and variable (20–100 steps). At the transition point, episode lengths lock to the maximum (200 steps) within 1–2 episodes. In seed 256 at $\alpha = 4.0$, episode length jumps from a mean of approximately 50 steps to sustained lengths of 150–200 steps at episode 189, exactly coinciding with the entropy collapse from $H \approx 3.5$ to $H \approx 1.6$ and the emergence of 45%+ per-episode validity rates. Entropy collapse causally precedes the episode length increase by a small number of episodes.

This suggests that phase transitions in this system are not gradual—they are sharp bifurcations in the agent's behavioral mode, visible simultaneously in entropy, validity rate, quality, and episode length.

4.4 The 2,000-Episode Escape

To study zombie escape dynamics, we ran CDE at $\alpha = 4.0$, seed 42 for 2,000 episodes:

Table 2: Zombie escape trajectory (CDE, $\alpha = 4.0$, seed 42, 2,000 episodes).

Episode	H	Entropy Frac	Valid/ep	Cumul Valid%	Status
0	4.41	0.905	0%	0.0%	Random
600	2.08	0.667	10.5%	1.08%	Zombie
1200	1.47	0.617	0%	1.43%	Cracking
1500	2.71	0.842	0%	2.41%	*Self-correction: zombie restored*
1700	1.12	0.403	47.3%	5.29%	Breaking through (2nd attempt)
1999	0.33	0.158	70.7%	18.97%	**Escaped**

The escape sequence reveals a tug-of-war between the quality signal (pushing toward valid commands) and the entropy self-correction mechanism (pushing entropy back up toward the zombie level). At episode 1,200, entropy drops to 1.47—the first escape attempt. By episode 1,500, the CDE's self-correction mechanism has recovered entropy to 2.71, restoring the zombie. The second attempt at episode 1,700 succeeds because REINFORCE has accumulated enough gradient signal from intermittent valid commands to overpower the entropy recovery.

Comparing with the $\alpha = 0$ baseline at the same seed: the phase transition occurs at approximately episode 400 versus episode 1,700—a **4.25× delay** imposed by the zombie attractor. The final valid rate is also lower (18.97% vs. 41.36% at $\alpha = 0$ through 2,000 episodes), indicating persistent drag from the entropy bonus even after escape.

4.5 Zombie Taxonomy

Our experiments reveal three distinct zombie subtypes, distinguished by their maintenance mechanism and escape dynamics:

Entropy Self-Correction Zombie (CDE decoder). Entropy is maintained at $H \approx 2$–3 by the CDE's bidirectional coupling (Section 5). The agent generates diverse gibberish with healthy-looking metrics. *Escapable*: quality signal from valid commands accumulates over many episodes. Escape timing is seed-dependent (episode 150–1,700 at $\alpha = 4.0$).

Absolute Zombie (ODE decoder, rich environment). No entropy recovery mechanism. Entropy collapses or drifts aimlessly. The agent is permanently stuck. Confirmed over 2,000 episodes at seed 42: 33/90,789 valid commands (0.04%), with valid rate actually *declining* over time.

Environmental Zombie (any decoder, sparse environment at high α). Weaker because the quality gap between valid and invalid commands is larger in sparse environments. Escapable by reducing error diversity. On the Steam Deck's sparser Linux shell, *all* conditions produced phase transitions—including the "absolute" ODE zombie.

This taxonomy reveals that the absolute zombie is not a property of the ODE decoder alone—it is a property of the ODE decoder *in a rich-output environment*. Reduce shell output diversity and the zombie breaks.

5 Entropy Self-Correction

The CDE decoder (Equation 5) exhibits a self-correcting entropy recovery mechanism absent in all other decoder types tested. We characterize this mechanism and its dual role in learning dynamics.

5.1 Mechanism

The CDE decoder's dynamics are governed by two coupled components: the autonomous drift $f_\theta(h)$ and the character feedback $g_\theta(h, c_{\text{prev}})$. When entropy collapses, the decoder concentrates on a small subset of characters. This means g_θ receives nearly constant input (the same few characters repeatedly), reducing its effective contribution. The autonomous drift f_θ then dominates, pushing the hidden state h toward its natural attractor—which, absent feedback, produces a more uniform character distribution. Entropy recovers. As entropy recovers, g_θ receives diverse input again, character feedback resumes, and the cycle repeats.

This creates a sustained oscillation rather than a single recovery event:

CDE (d=32, s7, α=4) : Step 15K: $H = 4.24 \rightarrow$ Step 18K: $H = 2.08$ (collapse)
$\rightarrow$ Step 21K: $H = 2.20$ (recovery) $\rightarrow$ Step 40K: $H = 1.82$ (2nd collapse)
$\rightarrow$ Step 42K: $H = 2.03$ (2nd recovery)

Each collapse-recovery cycle creates an exploration opportunity. During recovery phases, the decoder explores diverse character distributions, increasing the probability of generating valid commands. Over many cycles, REINFORCE gradually amplifies the quality differential.

5.2 Quantitative Comparison: CDE vs. ODE

Table 3: CDE vs. ODE entropy trajectories at $\alpha = 4.0$, seed 7, 300 episodes.

Episode	CDE (self-correcting)	ODE (no correction)
0	4.38	4.39
50	3.32	2.96
100	2.87	3.03
150	2.82	2.66
200	**3.40** (recovery)	2.39
250	2.47	**2.00** (no recovery)
275	**3.12** (recovery)	2.51
299	3.07	2.40
Entropy decline rate:	CDE -0.0075/100 eps	ODE -0.337/100 eps (**45× faster**)

The CDE decoder's entropy oscillates with clear recovery events at episodes 200 and 275. The ODE decoder monotonically declines from 4.39 to 2.40 with no recovery. The entropy decline rate differs by a factor of 45×.

5.3 The Double-Edged Sword

Entropy self-correction is beneficial at $\alpha = 0$ but harmful at $\alpha > 0$:

Table 4: Effect of entropy self-correction across regimes. Phase transition (PT) scorecard across 3 seeds.

	CDE α=0	ODE α=0	CDE α=4	ODE α=4
Seed 7	**19.8%**	0.1%	0.02%	0.1%
Seed 256	**19.6%**	0.4%	**29.2%**	0.0%
Seed 42	0.1%	0.1%	0.1%	**9.8%**
PT count	**2/3**	0/3	1/3	1/3

At $\alpha = 0$, CDE achieves 2/3 phase transitions while ODE achieves 0/3. The self-correction mechanism is *necessary* for valid command discovery without entropy bonus—it provides the repeated exploration opportunities that ODE's permanent collapse eliminates. At $\alpha = 4.0$, the picture inverts: CDE's recovery pushes entropy *back up* to the zombie level, maintaining the degenerate equilibrium more strongly than ODE. The seed 42 ODE anomaly ($\alpha = 4.0$ transition at 9.8%) suggests that the entropy bonus occasionally acts as a diversity preservation mechanism that enables discovery even without self-correction—but this is rare and seed-dependent.

6 Phase Transitions and Developmental Trajectory

6.1 The 9-Stage Developmental Trajectory

The full encoder-decoder system, trained with command length growing by +1 every 1,000 episodes, exhibits a reproducible developmental sequence:

1. **Random exploration** (EP 0–100): The agent generates random byte sequences. Entropy is near-maximum ($H \approx 4.4$). No valid commands.

2. **Phase transition** (EP 100–150): Valid commands are discovered. Entropy collapses sharply. The timing is seed-dependent (EP 114–350).

3. **Valid command lock-in** (EP 150–1,000): The agent concentrates on discovered commands (`dd`, `ld`, `id`, `df`, `ls`, `ln`). Validity rate climbs to 50%+.

4. **Meta-stability** (EP 1,000–1,500): Validity plateaus. The agent has exhausted novelty from known commands.

5. **Separator discovery** (EP 1,500–2,000): Characters like `/`, `<`, `>`, `&` are discovered as command length grows to 3+.

6. **Space discovery** (EP 2,000–2,500): The space character emerges, enabling argument passing.

7. **Path and flag discovery** (EP 2,500–5,000): Commands like `ls //`, `ls -x` appear.

8. **The Great Divergence** (EP 5,000–8,000): Strategies diverge across seeds—space-padding vs. redirect chains vs. path exploration.

9. **New command discovery** (EP 8,000–12,100+): Novel commands emerge via mutation buffer at higher command lengths (up to 14 characters).

The champion run (seed 137) sustained 12,100+ episodes, generating 39,593 commands with 95%+ final validity, quality reaching 9.03, and 9 unique valid commands discovered. Cross-platform reproduction confirmed the phase transition on macOS (50× growth in episode length) but not on Linux/Steam Deck (flat at ∼65 steps through 500 episodes), highlighting the role of shell output diversity.

6.2 Episode Length During Phase Transitions

Episode length exhibits a sharp bifurcation at phase transitions in the encoder-decoder system. Before transition, episode lengths are variable (20–100 steps, mean ∼30). At the transition point, lengths lock to near-maximum (150–200 steps) within 1–2 episodes. Over longer runs, the 1-step fixation rate (episodes terminating after a single step) increases gradually from 19% (early) to 35% (at 5,000 episodes), suggesting progressive entropic pressure without curriculum intervention.

The episode length dynamics are particularly informative because they are *not* directly rewarded—the agent receives no bonus for longer episodes. The length increase is a side effect of the agent discovering valid commands that produce rich output, which in turn produces high observation novelty, which sustains the episode (entropy stays above the termination threshold). The encoder is implicitly rewarded for longer episodes through accumulated novelty, creating a survival pressure that drives the developmental trajectory.

6.3 Significance of Phase Transitions

The phase transitions observed in this system are not merely performance improvements—they are qualitative changes in the agent's behavioral mode, analogous to phase transitions in physical systems. Before the transition, the agent exists in a high-entropy, low-validity regime where its character distribution is nearly uniform. After the transition, it occupies a low-entropy, high-validity regime where it has concentrated on a small set of character patterns corresponding to valid commands.

The transition is sharp (occurring over a few episodes or a few thousand steps), irreversible under the same conditions, and accompanied by characteristic signatures in multiple metrics simultaneously (entropy collapse, validity rate jump, episode length bifurcation, quality surge). As we show in Section 10, these transitions also have distinctive geometric signatures in weight space.

7 Error Diversity as Implicit Entropy Regularization

One of our most consequential findings is that the shell environment's error diversity acts as an implicit entropy regularizer—and a strictly superior one to explicit entropy bonuses.

7.1 Quantifying Error Diversity

Table 5: Shell environments and their error characteristics.

Shell Mode	Error Categories	% Identical Responses	Platform
Whitelist	1 ("command not allowed")	99.6%	All
Blacklist (macOS)	5+ (not-found, syntax, usage, empty, other)	∼12%	Mac
Blacklist (Ubuntu)	Similar but sparser PATH	∼20%	EC2
Blacklist (SteamOS)	Similar but sparser PATH	∼25%	Deck

In whitelist mode, 99.6% of invalid commands produce the identical response "command not allowed." The observation novelty for invalid commands is therefore near-zero after the first few steps, creating a steep quality differential between valid and invalid commands—but also requiring entropy collapse before valid commands can emerge (the agent must concentrate to escape the noise floor).

In blacklist mode, invalid commands produce diverse error messages across multiple categories. This diversity provides moderate observation novelty even for invalid commands, meaning the agent does not need to collapse entropy to achieve reasonable reward.

7.2 Impact on Phase Transitions

Table 6: Error diversity effect on CDE decoder ($\alpha = 0$, REINFORCE+baseline).

Shell Mode	Valid%	H at Transition	Mechanism
Whitelist	1.8–13% (seed-dependent)	H collapses to $\sim$1.8	Concentration required
Blacklist	**27–36%** (at $\sim$15K steps)	H **stays** $\sim$**4.2**	No collapse needed

This is a striking result: in blacklist mode, valid commands emerge at 27%+ validity *while maintaining near-maximum character entropy* ($H \approx 4.23$ out of a theoretical maximum of $\log_2(97) \approx 4.4$ bits). The CDE decoder achieves 4,336 valid commands in 14K steps on blacklist versus 13 valid commands in 50K steps on whitelist—a **334**× improvement. The environment's error diversity provides exactly the exploration signal that entropy regularization attempts to provide, but without creating the zombie attractor.

7.3 The Dual Role of Error Diversity

Error diversity interacts differently with the entropy bonus at different α values:

- **At** $\alpha = 0$: Error diversity *accelerates* phase transitions by providing richer gradient signal through varied error bigrams. The decoder receives different observation novelty values for different types of errors, creating a smoother optimization landscape.
- **At** $\alpha > 0$: Error diversity *sustains* the zombie attractor by providing observation novelty without valid commands. Diverse errors become the zombie's food—the agent can collect both entropy reward and moderate observation novelty from gibberish, eliminating the pressure to discover valid structure.

7.4 Cross-Platform Evidence

The role of environmental diversity is confirmed by cross-platform experiments:

Table 7: Cross-platform comparison (CDE, seed 7, 2,000 episodes).

Platform	Error Diversity	CDE α=0	CDE α=4	ODE α=0	Zombie Status
Mac (macOS)	High	*running*	0.02% (H=3.07)	0.1%	Strong zombie
Deck (SteamOS)	Low	34.7% (H=0.27)	35.2% (H=0.75)	22.4% (H=0.30)	**No zombie**

On the Steam Deck's sparser Linux shell, *all* conditions produce phase transitions—including the ODE at $\alpha = 0$ (which never transitions on macOS) and the CDE at $\alpha = 4.0$ (which is trapped in a zombie on macOS). The quality gap between valid and invalid commands is simply larger when errors are less diverse, making the signal easier for REINFORCE to amplify.

Ubuntu EC2 provides an intermediate case: REINFORCE achieves 5.1% valid on blacklist (vs. 0.8% on Mac blacklist), suggesting that moderate error diversity outperforms both extremes for REINFORCE-based optimization.

7.5 Against Explicit Entropy Regularization

Our evidence builds a case against explicit entropy regularization ($\alpha > 0$) in observation-novelty systems:

1. Observation novelty *replaces* entropy bonus entirely—all successful transitions in both phases use $\alpha = 0$.

2. Entropy bonus creates the zombie attractor, where valid commands are actively *punished* because they have lower character entropy than random gibberish.

3. CDE blacklist achieves 27% valid at $H = 4.27$—near-maximum entropy without any explicit entropy term.

4. Entropy self-correction provides architectural entropy regulation without the pathological failure mode.

5. The environment itself provides implicit entropy regularization through error diversity, if that diversity is structured appropriately.

Entropy regularization treats a symptom (premature entropy collapse) with a side effect worse than the disease (the zombie attractor). Architectural self-correction plus observation-grounded novelty provides a cleaner solution.

8 Decoder-Only Results

8.1 Architecture Comparison: Whitelist Mode

Table 8: Decoder architecture comparison (whitelist, $\alpha = 0$, best result per decoder, 50K steps).

Architecture	Coupling	Params	Best Valid%	Phase Transition
TDE	Tridirectional (ODE + feedback + memory)	20,964	**29.4%**	Sharp (PCA L-turn)
ODE	None (pure continuous trajectory)	7,490	13.8%	Slow
CDE	Bidirectional (ODE + feedback)	16,866	13.0%	Gradual (oscillations)
Transformer	Discrete self-attention	16,738	2.1%	Weak
SDE	Stochastic noise	8,546	0.1%	None

The TDE decoder, with its fast-weight memory enabling tridirectional coupling (ODE dynamics $\leftrightarrow$ character feedback $\leftrightarrow$ memory), achieves the highest validity by a wide margin. Its memory enables a form of within-episode credit assignment: the $d \times d$ Hebbian memory matrix accumulates information about which character patterns were recently generated, allowing the decoder to "remember what worked" across the character sequence.

The ODE and CDE decoders achieve similar validity (13.8% vs. 13.0%) but through different mechanisms. ODE relies on phase-space exploration without recovery; CDE uses its entropy self-correction oscillations. The Transformer baseline (2.1%) demonstrates that discrete attention mechanisms do not benefit from the observation novelty signal in the same way. SDE's learned noise (0.1%) proves insufficient—the noise is too unstructured to drive discovery.

8.2 Architecture Comparison: Blacklist Mode

Table 9: Blacklist results ($d = 32$, lr $= 10^{-4}$, seed 42, $\alpha = 0$, REINFORCE+baseline).

Decoder	Steps	Valid%	Key Observation
CDE	16.8K	**27.3%** (H=4.23)	No entropy collapse
ODE	11.4K	36.0% (H=4.38)	ODE benefits most from error diversity
Transformer	13K	0.8% (H=4.22)	Does not benefit from error diversity

A surprising result: the ODE decoder, which struggles in whitelist (13.8%), *outperforms* CDE on blacklist at 36.0%. The diverse errors provide the exploration signal that the ODE's missing entropy self-correction would otherwise provide. This confirms that error diversity and architectural self-correction are *substitute mechanisms* for the same function: maintaining sufficient exploration pressure.

8.3 Encoder vs. Decoder-Only

The decoder-only experiments demonstrate that the encoder is not essential for basic command discovery:

Table 10: Encoder-decoder vs. decoder-only comparison.

Configuration	Params	Valid%	Max cmd_len	Unique Cmds
FW-ODE encoder + CDE decoder	136K	95%+	14 chars	7,170
TDE decoder only (whitelist)	21K	29.4%	2 chars	7,583
CDE decoder only (blacklist)	17K	27.3%	2 chars	4,167

For basic 2-character command discovery, the decoder alone suffices. The encoder's value appears in three areas: (1) longer commands (`cmd_len` > 2) where sequential context matters, (2) sharper final concentration ($H = 0.27$ vs. $H = 4.27$ or $H = 1.16$), and (3) zombie escape at $\alpha = 4.0$.

The third point is critical: without the encoder, the decoder at $\alpha = 4.0$ is permanently trapped (0.0% valid through 2,000 episodes). With the encoder, the time-varying latent state z_{post} creates different decoder optimization landscapes at each step, allowing quality gradients to accumulate from multiple directions rather than being consistently dominated by entropy gradients. The encoder acts as a *gradient amplifier*: its changing state diversifies the gradient signal, enabling escape from the zombie attractor even when the entropy bonus is strong.

9 RL Algorithm Ablation

9.1 REINFORCE vs. PPO

Table 11: RL algorithm comparison across shell environments (CDE, $d = 32$).

	Whitelist (20K steps)	Blacklist (∼15K steps)
REINFORCE + baseline	0–7.3% (seed-dependent)	0.8%
PPO (clip = 0.2, 1% ent. coef.)	0.1%	**5.4%**
Raw REINFORCE (no baseline)	**0.0%**	—

The interaction between RL algorithm and environment is notable. REINFORCE with baseline outperforms PPO on whitelist (7.3% vs. 0.1% at best seed) but PPO outperforms REINFORCE on blacklist (5.4% vs. 0.8%). This suggests that sparse error environments require sharp policy concentration (REINFORCE's strength), while rich error environments benefit from sample efficiency and stable updates (PPO's multi-epoch optimization with 4 epochs per batch of 32 steps).

A caveat: our PPO implementation includes a small entropy coefficient (0.01), which is itself a form of entropy regularization. An isolation experiment (`ppo_entropy_coef` $= 0$) was planned but not completed due to compute constraints. The PPO blacklist advantage may partly reflect this implicit entropy bonus.

9.2 The Necessity of Baselines

Raw REINFORCE (without EMA baseline) produces 0% valid commands across all configurations. The baseline provides variance reduction that is essential for amplifying the 4× Jaccard differential between valid and invalid commands. Without it, the gradient signal is too noisy for REINFORCE to concentrate on productive character patterns.

10 PCA Analysis of Weight Trajectories

To understand the geometric structure of learning, we performed principal component analysis on decoder weight snapshots taken every 1,000 steps across 50,000-step training runs.

10.1 TDE: Discrete Phase Transition

The TDE decoder (29.4% valid, whitelist, $d = 32$, seed 42) exhibits a sharp, discrete phase transition visible in weight space:

- **PC1 explains 70.9%** of all weight variance—the transition dominates the entire learning process.
- Top 3 PCs explain 88.7% of variance (PC2: 12.5%, PC3: 5.2%).
- The weight trajectory traces an **L-shaped turn** in PC1–PC2 space at step 22,000–24,000, exactly coinciding with the phase transition.
- Prior to step 22,000: PC1 moves gradually from -1.70 to -0.37 while PC2 rises from -0.85 to $+0.78$. This is the "approach" phase.
- At step 22,000–23,000: **weight step size spikes to** $\Delta w = 0.743$ (1.7× the pre-transition average of 0.45), indicating a discrete restructuring of the network.
- After step 24,000: PC1 continues rightward to $+1.81$ while PC2 reverses downward to -0.68. Validity climbs linearly at approximately 0.6% per 1,000 steps.

The L-turn signature is interpretable: the TDE's fast-weight memory enables a form of "commitment." Before the transition, the network gradually moves toward a decision boundary. At the transition, the memory crystallizes a pattern that produces valid commands, and the network pivots sharply to exploit this discovery. The high PC1 variance (70.9%) indicates that this single event—the phase transition—accounts for the majority of all learning that occurs.

10.2 CDE: Continuous Phase Transition

The CDE decoder (1.85% valid, whitelist, $d = 32$, seed 7) exhibits a qualitatively different trajectory:

- **PC1 explains 64.6%** of weight variance—less dominated by a single event than TDE.
- Top 3 PCs explain 85.3% (PC2: 15.2%, PC3: 5.5%).
- The weight trajectory **curves smoothly** through PC space, with no sharp L-turn.
- Entropy oscillates between 1.8 and 2.3 in the post-transition regime (H at step 33K: 1.99, step 37K: 2.08, step 39K: 1.89, step 41K: 2.23), reflecting the entropy self-correction oscillation cycles.

- Weight step size shows a moderate peak at step 32,000–33,000 ($\Delta w = 0.558$, vs. a pre-transition average of 0.41), but no sharp spike.
- Validity climbs gradually from 0.12% at step 20,000 to 1.85% at step 50,000—approximately 0.06% per 1,000 steps, an order of magnitude slower than TDE.

The smooth trajectory reflects the CDE's oscillatory learning dynamics: each entropy self-correction cycle pushes the weights slightly further toward valid command generation, but no single cycle produces a decisive restructuring. The lower PC1 dominance (64.6% vs. 70.9%) confirms that learning is distributed across more independent weight movements rather than concentrated in a single transition event.

10.3 Comparative PCA Signatures

Table 12: PCA signatures across decoder architectures.

Decoder	PC1	PC2	PC3	Top-3	Transition Type
TDE ($s42$, 29.4%)	**0.709**	0.125	0.052	**0.887**	Sharp L-turn, Δw spike
TDE ($s7$, low)	0.595	0.150	0.067	0.812	Weak
CDE ($s7$, 1.85%)	0.646	0.152	0.055	0.853	Smooth curve
CDE ($s42$, low)	0.642	0.144	0.058	0.843	Smooth curve

The PCA signature is predictive of transition quality: higher PC1 dominance correlates with sharper transitions and higher final validity. The TDE at seed 42 (PC1 = 0.709, 29.4% valid) shows the sharpest transition, while the same architecture at seed 7 (PC1 = 0.595, low validity) shows a weaker signature—suggesting that the seed determines whether the initial weight configuration is positioned to undergo a strong transition.

10.4 Significance

The PCA analysis provides geometric evidence for two qualitatively distinct learning dynamics:

1. **Discrete transitions** (TDE): A critical restructuring event in weight space, mediated by memory, that resembles a first-order phase transition. The weight trajectory has a clear "before" and "after" with a sharp boundary.
2. **Continuous transitions** (CDE): A gradual accumulation of small weight changes driven by oscillatory entropy dynamics, resembling a second-order phase transition. No single event dominates; learning emerges from the superposition of many oscillation cycles.

These signatures may generalize beyond our specific setting: any system with memory-augmented policies might exhibit L-turn transitions, while systems with self-correcting dynamics might exhibit smooth trajectories. The PCA framework provides a diagnostic tool for identifying which regime a system occupies.

11 Discussion: Validity and High Entropy as Side Effects

11.1 The Quasi-Novelty Principle

The central finding of this work is that valid shell syntax emerges as a side effect of diversity pressure, not as a directly optimized objective. We call this *quasi-novelty* because the reward

signal is a shallow structural proxy (bigram Jaccard distance) that measures surface-level variation rather than semantic correctness or meaningful exploration. Yet this quasi-novelty signal is sufficient to drive the emergence of genuinely complex behavior—valid commands, command chaining, path discovery, flag usage—because the environment's structure creates a reliable gradient from surface diversity toward behavioral validity.

The zombie attractor embodies the other face of quasi-novelty. In the zombie regime, the agent achieves *apparent* novelty (high bigram diversity, varied character output) without any real discovery. It is quasi-novel in the pejorative sense: it appears to explore but produces nothing of value. The transition from zombie to productive agent is the transition from apparent to genuine quasi-novelty, mediated by the environment's structure.

This principle suggests that reward signals need not directly encode the target behavior—they need only correlate with it through the structure of the environment. The strength of this correlation (the quality differential) determines the ease and reliability of emergence.

11.2 High Entropy as Side Effect

In blacklist environments, the CDE decoder discovers valid commands while maintaining near-maximum entropy ($H = 4.23$, approximately 96% of the theoretical maximum). This contradicts the common assumption that task performance requires entropy reduction. In this regime, the agent generates valid commands from a broad character distribution because the environment's diverse responses make this strategy optimal for observation novelty. High entropy is not a side effect of failure (as in the zombie) but of success in a rich environment.

11.3 What the Results Mean

Taken together, our findings suggest several broader points:

Environment design matters as much as agent design. The same agent (CDE, $\alpha = 0$) achieves 0% validity on macOS blacklist with REINFORCE but 34.7% on Steam Deck. The same architecture achieves 13% on whitelist but 27% on blacklist. The environment's error structure is not background noise—it is a critical determinant of learning dynamics.

Entropy regularization is not a free lunch. In systems with observation-grounded reward signals, explicit entropy bonuses create failure modes (the zombie attractor) that are worse than the problem they solve (premature entropy collapse). Architectural solutions (entropy self-correction) and environmental design (appropriate error diversity) are cleaner alternatives.

Phase transitions are the fundamental unit of learning in this system. Validity does not emerge gradually—it appears through sharp bifurcations that simultaneously reorganize entropy, episode length, quality, and weight structure. Understanding when and why these transitions occur (or fail to occur) is more informative than tracking any single metric.

12 Limitations

We document limitations transparently, as this work is part of an ongoing investigation.

Seed sensitivity. Results are highly seed-dependent. TDE at seed 42 achieves 29.4% validity while TDE at seed 7 achieves 0.1%—a 294× difference. CDE at seed 7 achieves 13% while CDE at seed 42 achieves 3.5%. Phase transitions in the encoder-decoder system occur at 2/5 seeds at $\alpha = 0$. More seeds are needed for robust statistical claims.

Incomplete ablations. The full experimental tensor (4 decoders $\times$ 4 α values $\times$ 4 window sizes $\times$ 3 learning rates $\times$ 3 d_{model} $\times$ 3 seeds) comprises 1,728 combinations; we completed approximately 50 (2.9%). Critical gaps include:

- **Jaccard window size**: Only $W = 5$ and $W = 10$ were tested in the decoder-only experiments. Window sizes of 20 and 50 were not tested at all. Furthermore, the interaction between window size and shell mode (whitelist vs. blacklist) was not explored. We hypothesize that $W = 20$ may be effective in whitelist mode (where the novelty floor is very low and a longer memory helps maintain pressure) while $W = 5$ may be better suited to blacklist mode (where error diversity already provides a rich novelty landscape), but this remains untested.
- **Entropy bonus sweep**: Only $\alpha = 0$ and $\alpha = 4.0$ were tested in the decoder-only tensor. Intermediate values ($\alpha = 1, 2, 3$) and higher values ($\alpha = 8$) are absent, preventing us from characterizing the zombie attractor boundary precisely.
- **SDE noise scale**: Only the default noise scale (0.1) was tested. A systematic sweep over noise parameters might reveal configurations where SDE performs competitively.

Blacklist mode underrepresented. Only 4–5 of 81 decoder-only experiments used blacklist mode. All blacklist experiments used $W = 5$, $d = 32$, lr $= 10^{-4}$, seed 42. Cross-decoder validation in realistic shell environments is therefore sparse.

Compute and platform constraints. Experiments were conducted on a 2019 Apple Intel MacBook (macOS), a Steam Deck (SteamOS/Linux), and AWS EC2 Ubuntu instances. Blacklist experiments are bottlenecked by shell command execution time (each command requires a real subprocess call), making long runs expensive even with good compute. The EC2 instance was terminated before completing several planned experiments. No GPU acceleration was used.

Partial blacklist runs. Most blacklist experiments reached 13–17K of a planned 50K steps before being stopped or interrupted. Transition data is captured, but long-term dynamics remain uncharacterized.

PPO isolation. PPO experiments include a small entropy coefficient (`ppo_entropy_coef` = 0.01). The isolation experiment (= 0) was not completed, so we cannot cleanly separate PPO's clipping benefit from its implicit entropy bonus.

Encoder-decoder PCA. Weight snapshots exist for the encoder-decoder experiments (648 snapshots across 11 runs), but PCA was computed only for the decoder-only experiments. The encoder's weight trajectory during phase transitions—particularly the fast-weight memory dynamics—remains unanalyzed.

13 Future Directions

This work is part of a larger investigation into observation-grounded open-ended learning. Several directions are actively being pursued.

Latent-space novelty. The current formulation computes Jaccard distance over raw observation bigrams. A natural extension is to compute novelty in the encoder's latent space, measuring distance between hidden state representations rather than surface strings. This would make the novelty signal sensitive to abstract structural patterns rather than character-level variation.

Emergent inter-step planning. We observe early signs that multi-step command sequences (e.g., `cd` followed by `ls`) may emerge through the same novelty mechanism that drives single-command discovery. The sequential quality differential—novel output from `ls` after `cd /tmp` versus `ls` at root—creates a gradient that REINFORCE can exploit. The encoder's fast-weight memory, which carries 73% context across steps, provides the credit assignment mechanism. This would represent emergent planning as a side effect of survival pressure rather than explicit planning objectives.

Environment manipulation. The whitelist/blacklist dichotomy represents two extremes of error diversity. Intermediate regimes—controlled error vocabularies, tunable response diversity, structured error hierarchies—could reveal the optimal error landscape for learning. This is closely related to curriculum design for open-ended learning systems.

Scaling. Our agents are deliberately minimal (7–136K parameters). Whether the phenomena documented here—zombie attractors, entropy self-correction, phase transitions, quasi-novelty—persist, intensify, or vanish at larger scales is an open question.

14 Conclusion

We have presented a systematic investigation of emergent syntax discovery in minimal reinforcement learning agents trained purely on observation diversity. The Jaccard novelty formulation—a computationally cheap bigram-distance metric applied to shell output—serves as a sufficient reward signal for discovering valid Unix commands, achieving 95%+ validity in the full encoder-decoder system and 29.4% with a standalone 21K-parameter decoder, all without any reward for correctness.

The zombie attractor, a stable equilibrium in entropy-regularized systems, arises from the interaction of entropy bonus, environmental error diversity, and decoder architecture. It is not a permanent failure mode but a dynamical regime with specific entry conditions and, under architectures with entropy self-correction, specific escape dynamics. The entropy self-correction mechanism in CDE decoders creates oscillatory dynamics that are beneficial at $\alpha = 0$ (enabling exploration) but harmful at $\alpha > 0$ (maintaining the zombie).

Error diversity acts as an implicit entropy regularizer, enabling valid command discovery without entropy collapse and without the zombie pathology of explicit entropy bonuses. Phase transitions are sharp, multidimensional bifurcations with distinctive PCA signatures: L-turns in memory-augmented decoders, smooth curves in self-correcting decoders.

These findings are unified by the concept of quasi-novelty: a shallow diversity metric that, through the structure of the environment, induces the emergence of complex behavioral organization. The quasi-novelty principle suggests that sophisticated reward engineering may be less important than we think—if the environment is structured such that interesting behavior is also diverse behavior, a simple diversity pressure may suffice.

Acknowledgments

Experiments were conducted on personal hardware (2019 Apple Intel MacBook, Steam Deck) and AWS EC2 instances. This work received no external funding.